Съдържание

1 Стъпка 03 — Варианти на CFR и Монте Карло методи: Доклад за имплементацията 1
1.1 1. Въведение . . . 1
1.2 2. Основни концепции . . . 2
1.3 3. Описание на алгоритмите . . . 5
1.4 4. Фаза на изследване . . . 7
1.5 5. Имплементация: Времеви бенчмарк . . . 9
1.6 6. Анализ на сходимостта . . . 9
1.7 7. Обобщение и следващи стъпки . . . 11

1 Стъпка 03 — Варианти на CFR и Монте Карло методи: Доклад за имплементацията

Среда: Април 2026

Игра: Leduc Poker (6 карти, 2-ма играчи, 2 кръга на залагане)

Алгоритми: Vanilla CFR, CFR+, MCCFR External Sampling, MCCFR Outcome Sampling

Цели: Експлоатируемост на CFR+ < 0.001 в рамките на 180 сек., крос-валидация с OpenSpiel, анализ на сходимостта

Статус: Всички цели са постигнати

1.1 1. Въведение

Стъпка 02 установи работещ решавател чрез базов (vanilla) CFR за Kuhn Poker — минималния бенчмарк за несъвършена информация със само 12 информационни множества. Въпреки че е достатъчен за валидиране на рамката на CFR, базовият CFR изисква **пълно обхождане на дървото на играта при всяка итерация**. Този подход става неприложим с нарастването на игрите: Kuhn Poker има 58 терминални възела, докато Texas Hold'em има над 10^{14} информационни множества. Нито един компютър не може да изброи това дърво дори веднъж.

Стъпка 03 преодолява тази пропаст в мащабируемостта, като въвежда два допълващи се механизма:

1. **CFR+ (Tammelin, 2014):** Модификация на базовия CFR, която постига драстично по-бърза сходимост чрез лимитиране на съжалението (regret flooring), линейно усредняване на стратегиите и редуващи се обновявания. CFR+ беше алгоритъмът, който „разреши“ Heads-Up Limit Texas Hold'em — първият нетривиален вариант на покер, който беше практически решен.

2. **Monte Carlo CFR (Lanctot et al., 2009):** Вместо да обхожда цялото дърво на играта при всяка итерация, MCCFR семплира (взима извадка от) малка част — едно раздаване, един път. Всяка итерация е с порядъци по-евтина, с цената на по-шумни обновявания. Това е единственият практически приложим подход за игри, където пълното обхождане е физически невъзможно.

Двата алгоритъма са имплементирани от нулата върху **Leduc Poker**, бенчмарк от среден мащаб (~936 информационни множества, ~78× по-голям от Kuhn), който е достатъчно малък за точни изчисления, но и достатъчно голям, за да разкрие значими разлики в производителността между вариантите на CFR.

Връзка с дисертацията: Тази стъпка води директно до **Принос № 1 (Рамка за поведенческа адаптация)**. MCCFR е алгоритъмът, който изчислява базовата стратегия на Неш (Nash strategy) — „игра на сигурно“ котвата — от която адаптивният агент се отклонява въз основа на наблюденията на опонента. CFR+ прави това изчисление изпълнимо. Имплементацията на Leduc също така установява междинната бенчмарк игра, използвана в Стъпки 5–8 преди мащабирането до пълен покер.

Разработени компоненти:

- **Двигател (engine) за играта Leduc Poker:** Пълна имплементация на играта — 6 карти, 2 кръга на залагане, обща карта, действия fold/call/raise, 120 пермутации на раздавания, класиране на ръце базирано на чифтове. Напълно съвпада със семантиката на OpenSpiel.
- **Обучител (trainer) за Vanilla CFR:** С пълно обхождане и буферирано натрупване на съжалението за всички 120 раздавания.
- **Обучител за CFR+:** С лимитиране на съжалението в нулата, линейно претегляне на стратегията, редуващи се обновявания на играчите (Tammelin, 2014).
- **Обучител за MCCFR External Sampling:** Едно семплирано раздаване на итерация, всички действия на обхождащия се изследват, действията на опонента се семплират (Lanctot et al., 2009).
- **Обучител за MCCFR Outcome Sampling:** Единична семплирана траектория с ε-on-policy изследване и корекция на значимостта чрез importance sampling.
- **Оценител на експлоатируемост:** Итеративно изчисляване на най-добър отговор с ограничение по информационни множества.
- **Времева бенчмарк система:** Изпълнява всичките четири алгоритъма при общ бюджет за реално време с геометрично разпределени моментни снимки (snapshots).
- **Пакет за крос-валидация:** Сравнява резултатите с референтните имплементации на OpenSpiel.

1.2 2. Основни концепции

1.2.1 2.1 Търсене в дърво с Монте Карло (MCTS)

Търсенето в дърво с Монте Карло (Monte Carlo Tree Search - MCTS) е фамилия алгоритми, които използват **случайно семплиране** за оценка на позиции в дърво на игра без изчерпателно изброяване. MCTS изгражда асиметрично дърво за търсене постепенно, фокусирайки изчисленията върху най-обещаващите клонове.

Алгоритъмът следва четири повтарящи се фази:

1. **Селекция (Selection):** Започвайки от корена, се спуска надолу по дървото, като избира деца въз основа на политика за селекция (обикновено UCB1 — Upper Confidence Bound), която балансира експлоатацията на известни добри ходове с изследването на по-слабо посещавани такива.
2. **Експанзия (Expansion):** Когато бъде достигнат листовиден възел, той се разширява чрез добавяне на един или повече подвъзли към дървото.
3. **Симулация (Simulation/Rollout):** От новия възел се изиграва случайна игра до завършване — без стратегия, само равномерно случайни ходове до термиално състояние.
4. **Обратно разпространение (Backpropagation):** Резултатът (победа/загуба/равенство) се разпространява обратно нагоре през посетените възли, обновявайки техните статистики (брой посещения и кумулативно възнаграждение).

След много итерации, детето на корена с най-висок брой посещения (или най-високо средно възнаграждение) се избира като най-добър ход. Ключовото прозрение е, че **случайните симулации, агрегирани през хиляди итерации, клонят към точни стойностни оценки** — дори без никакво предварително познание от домейна отвъд правилата на играта.

MCTS е основата зад AlphaGo и AlphaZero, където фазата на случайна симулация е заменена с невронна мрежа. Подходът е най-подходящ за игри със **съвършена информация** (Го, Шах), но философията на семплиране директно вдъхновява Монте Карло методите, използвани в MCCFR за среди с несъвършена информация.

1.2.2 2.2 MCTS срещу Minimax

Minimax с алфа-бета отсичане (alpha-beta pruning) е класическият подход към търсене в дърво на игра. Той оценява **всеки** релевантен възел в дървото до фиксирана дълбочина, след което прилага евристична оценъчна функция в листата. Това работи добре, когато:

- Дървото на играта има управляем фактор на разклоняване (Шах: ~35, Дама: ~10).
- Съществуват добри оценъчни функции.
- Дълбочината на търсене може да бъде ограничена без загуба на критична информация.

MCTS възприема фундаментално различен подход: вместо да оценява всички клони до фиксирана дълбочина, той оценява **някои** клони до **пълна дълбочина** (терминални състояния). Сравнението:

Свойство	Minimax + α-β	MCTS
Покритие на дървото	Всички клони до дълбочина d	Избрани клони до термиално състояние
Оценка	Евристика при лимит на дълбочина	Точна (терминална функция) или невронна мрежа

Свойство	Minimax + α-β	MCTS
Чувствителност към разклоняване	Експоненциална: $O(b^d)$	Плавно градиране: фокусира се върху обещаващи клони
Нужни познания за домейна	Изисква силна оценъчна функция	Никакви (random rollout) или заучени
Най-подходящ за	Умерено разклоняване, добри евристики	Високо разклоняване, слаби/никакви евристики

За Го (където факторът на разклоняване е ~250), Minimax е практически неприложим. MCTS беше пробивът, който направи AI играчите на Го конкурентоспособни.

1.2.3 2.3 Маркови вериги и Монте Карло методи

"Монте Карло" в MCCFR се отнася до широка фамилия изчислителни методи, които използват **многократно случайно семплиране** за получаване на числени резултати.

Математическата основа се уповава на **Маркови вериги** (Markov chains) — стохастични процеси, при които следващото състояние зависи само от текущото състояние, а не от историята на достигането му (**Марково свойство**). Обхождането на дървото на играта може да се разглежда като Маркова верига: преходът зависи само от текущото състояние и избраното действие.

Ключовата теоретична гаранция е **Законът за големите числа**: ако семплираме достатъчно траектории през Марковата верига, средната стойност на семплираните стойности клони към истинската очаквана стойност. За MCCFR това означава:

$$\frac{1}{T}\sum_{t=1}^{T}\hat{v}_I^{(t)} \xrightarrow{T\to\infty} v_I$$

Семплираните стойности са **неизместени оценители (unbiased estimators)** — тяхната очаквана стойност е равна на истинската — което гарантира сходимост към същото равновесие на Неш като базовия CFR.

Компромисът е **вариацията (variance)**: отделните семпли могат значително да се отклоняват от истинската стойност, изисквайки повече итерации. Този компромис между вариация и скорост е централната тема в тази стъпка.

1.2.4 2.4 Математика на покера

Покерът съчетава три източника на сложност: 1. **Скрита информация:** Ограничава видимостта, създавайки информационна асиметрия. 2. **Стохастични елементи:** Раздаването на карти въвежда случайни възли. 3. **Стратегическа измама (блъфиране):** Равновесието на Неш изисква рандомизирано блъфиране с математически прецизна честота.

Формалната рамка използва **игри в разгърната форма** със следните компоненти: - **Дърво на играта:** Всички възможни последователности от действия и случайни събития. - **Информационни множества (Information sets):** Групи от състояния, неразличими за играча. - **Стратегии:** Съответствия от информационни множества към вероятностни разпределения на действията. - **Равновесие на Неш (Nash equilibrium):** Стратегически профил, при който никой играч не може да подобри очакваната си полезност чрез едностранна промяна.

Ключови величини: - **Очаквана стойност (Expected value - EV):** Средното възнаграждение. - **Най-добър отговор (Best response):** Оптималната контра-стратегия (най-силната експлоатация). - **Експлоатируемост (Exploitability):** Средната печалба на перфектния опонент.

1.2.5 2.5 Leduc Poker срещу Kuhn Poker

Leduc Poker значително мащабира сложността спрямо Kuhn:

Свойство	Kuhn Poker	Leduc Poker
Карти	3 (J, Q, K)	6 ({J, Q, K} × {♠, ♥})
Кръгове	1	2
Обща карта	Няма	1 разкрита между кръговете
Информационни множества	12	936
Възли в дървото	58	10,200
Случайни изходи	6	120

Въпреки че е ~78× по-голям от Kuhn, Leduc остава достатъчно малък за точни изчисления (пълно обхождане отнема ~50ms), което го прави идеален бенчмарк за сравнение.

1.3 3. Описание на алгоритмите

1.3.1 3.1 CFR+ (Regret Matching Plus)

CFR+ (Tammelin, 2014) прави три модификации на базовия CFR:

Модификация 1 — Лимитиране на съжалението (Regret Flooring):

В базов CFR кумулативните съжаления могат да станат произволно отрицателни. В CFR+ съжаленията се лимитират до нула (т.е. не могат да бъдат отрицателни) след всяко обновяване:

$$R^{T+1}(I,a) = \max\left(R^T(I,a) + r^T(I,a),\ 0\right)$$

Това предотвратява натрупването на голямо отрицателно съжаление, което изисква дълго време за "изплащане". Аналогията с ReLU активациите в невронните мрежи е директна.

Модификация 2 — Линейно усредняване на стратегиите:

Вместо да усреднява еднакво всички итерации, CFR+ претегля всяка итерация t с нейната стойност t:

$$\bar{\sigma}^T(I,a) = \frac{\sum_{t=1}^{T} t \cdot \sigma^t(I,a)}{\sum_{t=1}^{T} t}$$

По-късните итерации са по-добри и допринасят повече, което ускорява сходимостта.

Модификация 3 — Редуващи се обновявания:

Обновяването на играч 0 става на нечетни итерации, а на играч 1 — на четни. Това намалява работата наполовина и избягва едномременни смени в стратегиите.

Комбиниран ефект: Емпиричната скорост на сходимост се подобрява от $O(1/\sqrt{T})$ до приблизително $O(1/T)$ — драматично ускорение.

1.3.2 3.2 MCCFR External Sampling

External Sampling (Lanctot et al., 2009) заменя пълното обхождане с частично семплиране:

- **Случайни възли:** Семплира се едно раздаване на итерация.
- **Възли на обхождащия играч:** Изследват се **всички** действия (както при базов CFR).
- **Възли на опонента: Семплира** се едно действие спрямо текущата стратегия на опонента.

Всяка итерация засяга ~42 възли (спрямо 20,400 при пълно обхождане) — приблизително **945× по-бързо**. Заради дисперсията обаче External Sampling изисква стотици хиляди пъти повече итерации за постигане на същата експлоатируемост.

1.3.3 3.3 MCCFR Outcome Sampling

Outcome Sampling отвежда семплирането още по-далеч — изследва се само **една траектория**:

- **Случайни възли:** Едно раздаване.
- **Всички възли на играчи:** Семплира се едно действие.

- **Действия на обновявания играч:** Семплират се с ε-on-policy (с вероятност ε се избира равномерно).
- **Корекция (Importance sampling):** Тъй като действията се семплират, съжалението трябва да се коригира със съотношението между истинската вероятност и вероятността за семплиране.

Всяка итерация докосва само ~5.5 възли — **2,228× по-бързо**, но с още по-голяма вариация. Алгоритъмът е **най-евтиният, но и най-шумният** вариант на MCCFR.

1.4 4. Фаза на изследване

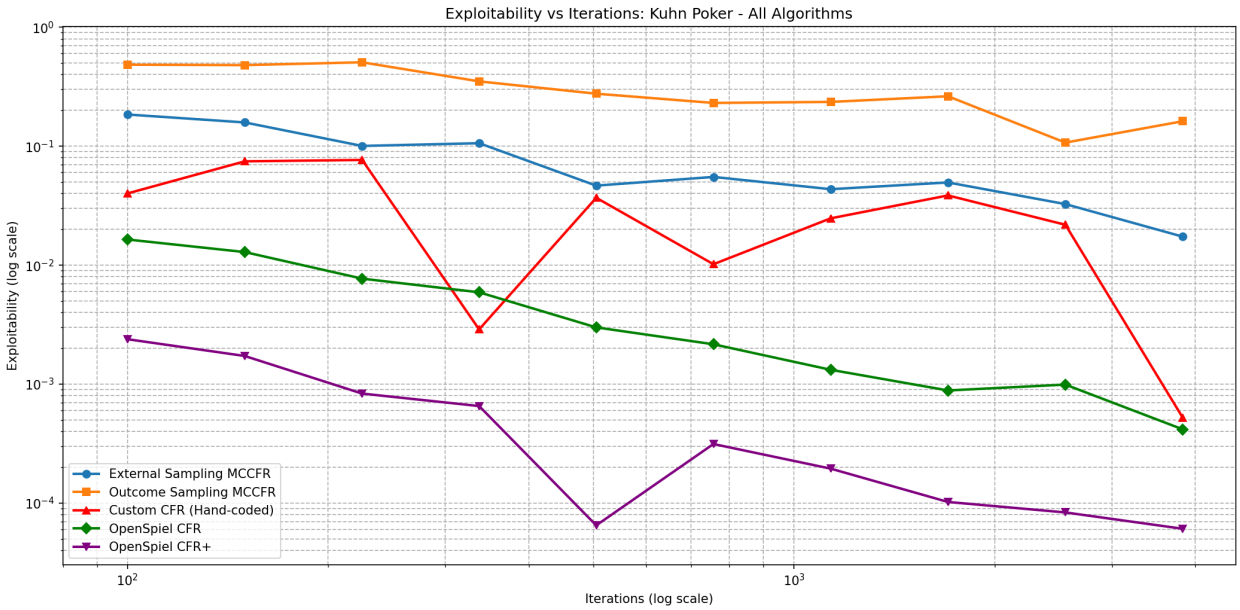
Преди имплементацията от нулата, бяха проведени предварителни експерименти чрез OpenSpiel.

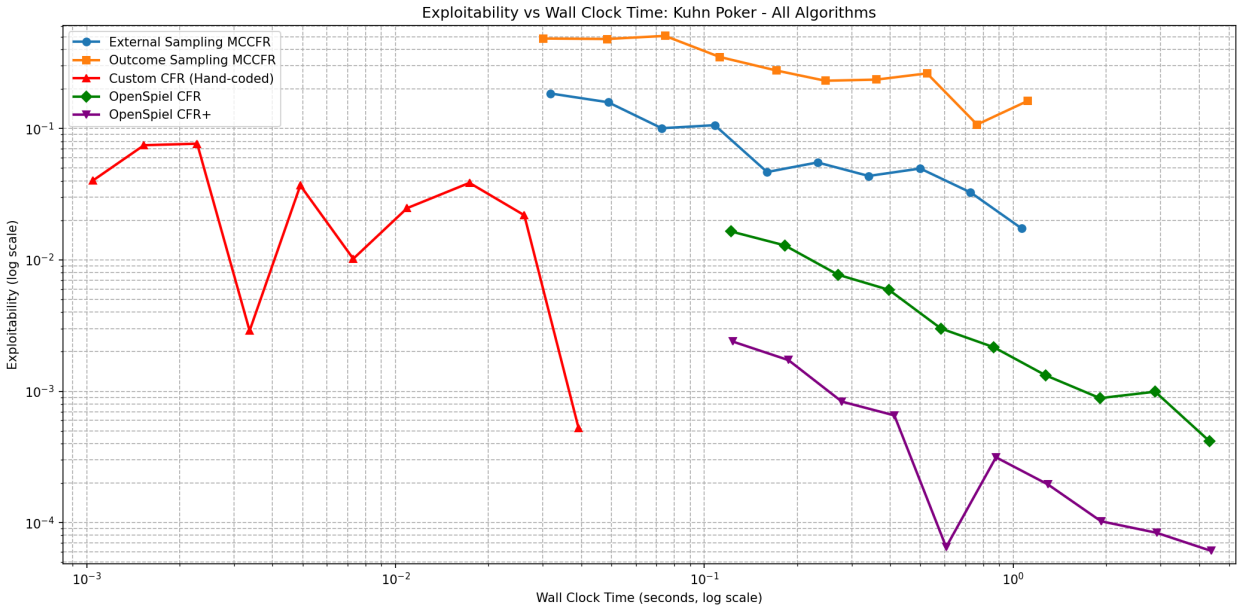
1.4.1 4.1 Експерименти с Kuhn Poker

Сравнение на 5 алгоритъма при 5,000 итерации:

Резултати на 5,000 итерации:

Алгоритъм	Експлоатируемост	Време
Собствен CFR (Стъпка 02)	~0.00035	< 1s
OpenSpiel CFR	~0.0015	~2s
CFR+	~0.0003	~2s
External Sampling MCCFR	~0.004	~1s
Outcome Sampling MCCFR	~0.025	~1s





Всички алгоритми сближават бързо; CFR+ е най-бърз.

1.4.2 4.2 Експерименти с Leduc Poker

На Leduc (936 информационни множества), 5,000 итерации:

Алгоритъм	Експл. @5k ит.	Време за 5k ит.
CFR+	~0.000054	~859s
Vanilla CFR	~0.0076	~747s
External Sampling MCCFR	~1.17	~7s
Outcome Sampling MCCFR	~3.08	~5s

Разликата тук е огромна - CFR+ достига нулева експлоатируемост, докато Outcome sampling е над 3.0.

1.4.3 4.3 5-минутно състезание

Алгоритъм	Експл. след 5 мин	Итерации
CFR+	~0.00042	~300
External Sampling	виж кеша	~200k+
Outcome Sampling	~0.527	~милиони

CFR+ е доминиращият алгоритъм за игри от такъв мащаб.

1.5 5. Имплементация: Времеви бенчмарк

Основната имплементация предоставя четири варианта на CFR от нулата върху Leduc Poker, тествани при общ бюджет от 180 секунди реално време за всеки. Целият код е написан ръчно на Python само с NumPy — без библиотеки за решаване на игри (game-solving libraries).

Разработени компоненти (написани ръчно в Python/NumPy): - **Двигател за Leduc Poker:** Пълна имплементация, съответстваща на семантиката на OpenSpiel. - **Vanilla CFR:** Пълно обхождане с буферирано натрупване на съжалението за всички 120 раздавания. - **CFR+:** Лимитиране до нула на съжалението, линейно претегляне, редуващи се обновявания. Буферираните съжаления се прилагат атомарно (atomically) при всяко преминаване. - **MCCFR External Sampling:** Едно семплирано раздаване на итерация. - **MCCFR Outcome Sampling:** Една семплирана траектория на итерация с ε-on-policy сърч (ε = 0.6). - **Най-добър отговор / Оценител на експлоатируемост:** Итеративно изчисляване на експлоатируемостта.

Резултати от 180-секундния бенчмарк:

Алгоритъм	Итерации	Крайна Експл.	Инф. множества
Vanilla CFR	3,713	0.0044	936
CFR+	3,706	0.000026	936
MCCFR External	3,504,665	0.0548	936
MCCFR Outcome	8,262,137	0.1030	936

1.5.1 Експлоатируемост спрямо Итерации (log-log)

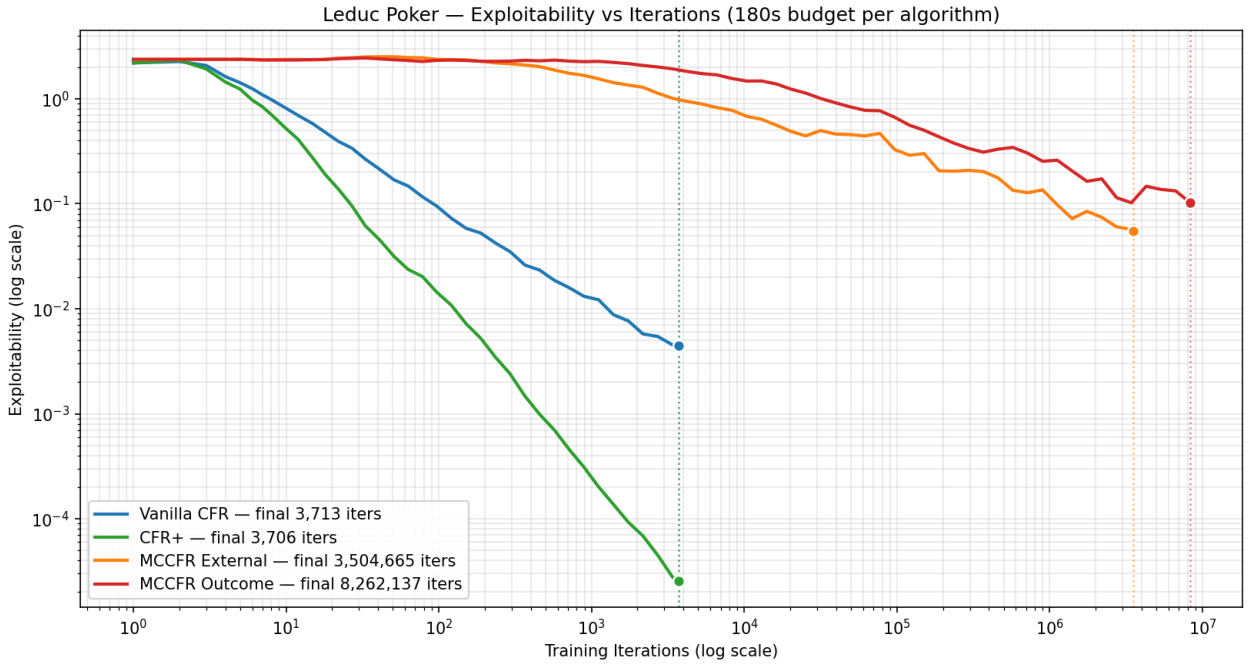
Пунктираните вертикални линии отбелязват финалния брой итерации за всеки алгоритъм. Вариантите, базирани на семплиране, изпълняват милиони итерации за 180 секунди, докато тези с пълно обхождане завършват ~3,700 — но всяка итерация с пълно обхождане е драматично по-информативна.

1.5.2 Експлоатируемост спрямо Време (log-y)

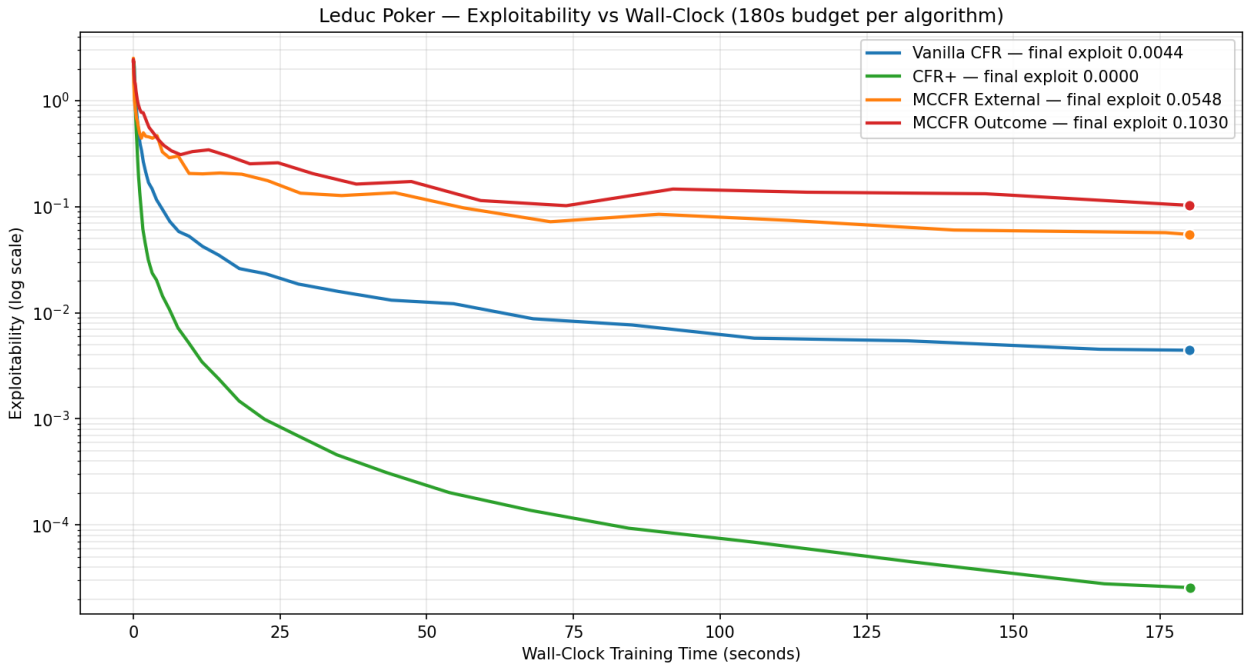
CFR+ доминира, достигайки експлоатируемост 0.000026 — почти точно равновесие на Неш — за 3 минути. Семплиращите варианти остават много по-назад при такъв размер на играта.

1.6 6. Анализ на сходимостта

Най-изненадващият резултат в тази стъпка е, че въпреки **1,000× повече итерации**, MCCFR External постига **12× по-лоша експлоатируемост** от Vanilla CFR.



Фигура 1: Експлоатируемост спрямо Итерации



Фигура 2: Експлоатируемост спрямо Време

1.6.1 6.1 Вариация спрямо скорост

Алгоритъм	C_{iter}	Скорост (it/s)	C_w	Спрямо Vanilla
Vanilla CFR	0.34	20.6	**0.075**	1.0×
CFR+	0.34	20.6	**~0.002**	~38× по-бърз
MCCFR External	107	19,470	**0.767**	10.2× по-бавен
MCCFR Outcome	245	45,901	**1.143**	15.3× по-бавен

Предимството в скоростта **не може да преодолее наказанието от вариацията**: MCCFR семплира по едно раздаване на итерация, което води до висока дисперсия поради скритата информация (кои карти са раздадени).

1.6.2 6.2 Формулата за пресичане (The Crossover Formula)

MCCFR става конкурентен, когато дървото на играта е достатъчно голямо:

Игра	$\lvert N \rvert$	External печели?	Outcome печели?
Leduc Poker	10,200	Не (210× твърде малко)	Не (466× твърде малко)
Limit Hold'em	~10^{14}	Да (10^7× над прага)	Да

Заключението е категорично: за малки до средни игри като Leduc, методите с пълно обхождане печелят. За Texas Hold'em (10^{14}+ възли), MCCFR е единственият практически възможен вариант.

1.7 7. Обобщение и следващи стъпки

1. **Четири CFR варианта** имплементирани от нулата.
2. **CFR+ потвърден като доминиращ алгоритъм** за малки и средни игри.
3. **Компромисът между вариация и скорост при MCCFR** е количествено измерен: External Sampling изисква ~100,000× повече итерации въпреки че е 945× по-евтин на итерация.
4. **Точката на пресичане (crossover point)** е изведена — MCCFR става конкурентен при ~2.1 милиона възли (External).

1.7.1 Следващи стъпки

- **Стъпка 04:** Абстракция на играта и мащабиране.
- **Стъпка 05:** Невронно равновесие (Deep CFR).